



Terraform

Interview Notes

200 Questions & Answers

Beginner • Intermediate • Advanced

Real Project + Interview Answers

Curated study guide • Azure focused



Table of Contents

01	Section 1: Core Concepts	6 Q
02	Section 2: Terraform Commands	17 Q
03	Section 3: State Management	15 Q
04	Section 4: Variables, Outputs & Locals	11 Q
05	Section 5: Modules & Reusability	9 Q
06	Section 6: Lifecycle & Dependencies	12 Q
07	Section 7: DevSecOps — tfsec, tflint, Checkov	5 Q
08	Section 8: CI/CD Integration	5 Q
09	Section 9: Advanced Concepts	20 Q
10	Section 10: Terraform Cloud & Enterprise	10 Q
11	Section 11: Scenario & Troubleshooting Questions	18 Q
12	Section 12: Real Project Answers	22 Q
13	Section 13: Tricky Conceptual Questions	10 Q
14	Section 14: Azure-Specific Terraform Questions	10 Q
15	Section 15: Pipeline & Workflow Deep Dive	8 Q
16	Section 16: Comparison Questions (Interviewer Favorites)	10 Q
17	Section 17: Security & Compliance Questions	6 Q
18	Section 18: Real Interview Scenario Questions	6 Q

Section 1: Core Concepts

Q1. What is Terraform?

Terraform is a declarative Infrastructure as Code (IaC) tool that manages the full lifecycle of cloud resources by maintaining a state file. It reads HCL configuration and interacts with cloud providers like Azure to create, update, and delete resources automatically.

■ **Tip:** Add: "In my project, I use Terraform to provision Azure resources — VMs, storage, networking — integrated with Azure DevOps for automated deployments."

Q2. What is Infrastructure as Code (IaC)?

IaC means managing infrastructure using code instead of manual steps — ensuring automation, consistency, version control, and repeatability across environments.

Q3. What does Terraform do?

Terraform automates infrastructure provisioning by reading configuration files and interacting with cloud providers like Azure to create and manage resources.

Q4. Why use Terraform?

It provides automation, consistency, scalability, and reduces manual errors. It is cloud-agnostic, supports hundreds of providers, and enables team collaboration through remote state.

Q5. What is a Provider?

A provider is a plugin (like `azurerm`) that allows Terraform to interact with cloud platforms. It translates HCL code into API calls for the cloud provider.

Q6. What is a Resource?

A resource represents a cloud component like a VM, storage account, or network — defined and managed in Terraform configuration.

Section 2: Terraform Commands

Q7. What is terraform init?

Initializes the working directory, downloads provider plugins, and sets up the backend for state storage. Always run this first.

Q8. What is terraform plan?

Creates an execution plan showing exactly what Terraform will do (Create, Update, or Delete) without actually making any changes.

Q9. What is terraform apply?

Executes the actions proposed in a Terraform plan to reach the desired state. In production, always use a saved plan file.

Q10. What is terraform destroy?

Removes all infrastructure managed by the specific Terraform configuration. Use with caution — add `prevent_destroy` lifecycle to protect critical resources.

Q11. What is terraform fmt?

Formats Terraform code for readability and standardization. Run before committing code.

Q12. What is terraform validate?

Checks syntax and configuration errors without connecting to the cloud — fast local check.

Q13. What is terraform refresh?

Updates the state file by comparing it with real infrastructure — detects changes made outside Terraform (drift).

Q14. What is terraform graph?

Shows the dependency graph of all resources in the configuration.

Q15. What is terraform console?

Interactive CLI to test expressions, functions, and resource references.

Q16. What is terraform show?

Displays details of the current state file or a saved plan file.

Q17. What is terraform output command?

Displays output values defined in the configuration after a successful apply.

Q18. What is terraform providers?

Lists all providers used in the current configuration.

Q19. What is terraform plan -out?

Saves the execution plan to a file so the exact same reviewed plan can be applied later — essential in CI/CD approval workflows.

Q20. Why use saved plan?

Ensures the same plan that was reviewed and approved gets applied — no surprises between plan and apply in a team environment.

Q21. What is terraform apply -auto-approve?

Skips manual approval and directly applies changes. NEVER use in production.

Q22. Why avoid auto-approve in production?

It can apply unintended changes without review — always use manual approval gate in production pipelines.

Q23. What is terraform init -upgrade?

Upgrades provider plugins to the latest version allowed by required_providers constraints.

Section 3: State Management

Q24. What is Terraform State?

A JSON file (terraform.tfstate) that maps your configuration to real-world resources. Terraform uses it to track what has been created and plan updates/deletions.

Q25. What is a Backend?

Backend defines where the state file is stored — locally or remotely. Always use remote backend in team environments.

Q26. What is Remote Backend?

Storing the state file in a shared remote location (like Azure Blob Storage) instead of a local machine — enables team collaboration and prevents state conflicts.

Q27. What is State Locking?

A mechanism that prevents multiple users from running Terraform simultaneously on the same state. In Azure backend, uses blob lease to lock.

■ *Tip: "In my project, we use Azure Blob Storage as remote backend with blob lease locking — prevents conflicts when multiple engineers work simultaneously."*

Q28. What is terraform state rm?

Tells Terraform to STOP managing a resource. The resource stays alive in Azure, but Terraform forgets it. Does NOT delete the resource from cloud.

Q29. What is terraform state mv?

Moves or renames a resource in the state file — used to restructure configuration without destroying and recreating resources.

Q30. What is the locking mechanism in Azure backend?

Uses blob lease to lock the state file — automatically locked during operations and released when done.

Q31. What happens if state is lost?

Terraform cannot track resources. Restore from backup (enable versioning on Azure Blob Storage) or re-import resources using terraform import.

Q32. What is the state file format?

Stored in JSON format — but never edit it manually. Always use terraform state commands.

Q33. What is the terraform state command?

Used to manage and inspect state — includes sub-commands: list, show, mv, rm, pull, push.

Q34. What is Drift in Terraform?

Drift occurs when infrastructure is changed manually outside Terraform — real state no longer matches configuration.

Q35. How do you handle drift?

Run terraform plan to detect drift, then terraform apply to restore desired state. For intentional changes, update configuration to match.

Q36. What is dependency lock file?

.terraform.lock.hcl ensures consistent provider versions across all team members and CI/CD runs. Always commit this file to Git.

Q37. What is required_version?

Defines the Terraform CLI version constraint — prevents running with incompatible Terraform versions.

Q38. What is required_providers?

Specifies provider version constraints — ensures consistent provider behavior across all environments.

Section 4: Variables, Outputs & Locals

Q39. What are Variables?

Variables make configurations reusable and flexible — pass different values for dev, test, and production without changing the core code.

Q40. Types of Variables?

String, number, bool, list, map, set, object, tuple.

Q41. What is output?

Outputs display useful information after execution — like IP addresses, resource IDs, or connection strings. Also used to pass values between modules.

Q42. What are Locals?

Local values simplify expressions and avoid repetition — internal computed values, not external inputs.

Q43. Difference between Variable and Local?

Variables are external inputs provided by users or pipeline. Locals are internal computed values defined within the module.

Q44. What is sensitive variable?

Prevents sensitive data (passwords, keys) from appearing in Terraform output, plan, or logs.

Q45. What is variable validation?

Restricts input values using conditions — for example, ensuring environment name is one of "dev", "test", "prod".

Q46. What is map vs object?

Map is key-value collection where all values have the same type. Object has fixed structure where each attribute can have a different type.

Q47. What is terraform.tfvars?

A file used to define variable values automatically — Terraform loads it without needing to specify it on the command line.

Q48. What is auto.tfvars?

Any file ending in .auto.tfvars is automatically loaded by Terraform without specifying it explicitly.

Q49. What is input variable precedence (lowest to highest)?

Environment variables (TF_VAR_name) — lowest priority terraform.tfvars terraform.tfvars.json *.auto.tfvars or *.auto.tfvars.json -var or -var-file CLI flags — always win (highest priority)

Section 5: Modules & Reusability

Q50. What are Modules?

Reusable blocks of Terraform code for better organization — like a VNet module that creates VNet, subnets, and NSGs, called from dev, test, and prod environments.

■ **Tip:** "In my project, I have created reusable modules for Azure VNet, Key Vault, App Service, VM, and storage."

Q51. Monolithic vs Modular Terraform?

Monolithic = single large file. Modular = organized reusable modules with clear inputs and outputs. Modular is always preferred for team projects.

Q52. What is Private Module Registry?

A secure internal place to store and version reusable Terraform modules — available in Terraform Cloud or Azure DevOps Artifacts.

Q53. What is versioning in modules?

Assigning versions to modules ensures stability — existing callers are not affected until they explicitly upgrade the version reference.

Q54. If multiple modules depend on each other?

Use output values from one module as input variables to another. Terraform automatically manages the order of operations.

Q55. What is your Terraform project structure?

modules/ — reusable modules (vnet, vm, keyvault, appservice, storage) environments/dev, test, prod — separate backend and variable files main.tf, variables.tf, outputs.tf — per environment backend.tf — remote state configuration per environment .terraform.lock.hcl — committed to version control

Q56. What is Terraform Registry?

Public repository where community providers and reusable modules are published. Use standard modules instead of writing everything from scratch.

Q57. What is module registry usage in project?

We use it to reuse standard modules instead of writing code from scratch — faster development and community-tested patterns.

Q58. What is provider version constraint?

Ensures a specific version of provider is used to avoid compatibility issues across environments and team members.

Section 6: Lifecycle & Dependencies

Q59. What is a Lifecycle Block?

Controls resource behavior during create, update, and delete — used to prevent accidental deletion, ensure zero-downtime updates, and manage replacement triggers.

Q60. What is `create_before_destroy`?

Creates the new resource before deleting the old one — ensures zero-downtime replacements for SSL certificates, VM images, etc.

Q61. What is `prevent_destroy`?

Safeguard that prevents a resource from being accidentally deleted. Terraform throws an error if destroy is attempted.

■ **Tip:** "I use `prevent_destroy` on Azure Key Vault and storage accounts to protect critical production resources."

Q62. What is `ignore_changes`?

Prevents Terraform from updating specific attributes even if they differ from configuration — useful when some properties are managed externally.

Q63. Use case of `ignore_changes`?

When Azure Policy sets tags automatically — we `ignore_changes` on the tags attribute to prevent Terraform conflicts.

Q64. What is `replace_triggered_by`?

Forces resource recreation when a specific referenced resource changes — useful for resources that do not detect all changes automatically.

Q65. What is `depends_on`?

Explicitly defines dependency between resources when Terraform cannot detect it automatically — ensures correct creation/destruction order.

Q66. When to use depends_on?

When Terraform cannot detect the dependency automatically — for example, when a script depends on a role assignment being completed first.

Q67. What is taint? (Deprecated)

Previously marked a resource for recreation on next apply. Now DEPRECATED — use terraform apply -replace="resource_address" instead. Note: Always mention this is deprecated — shows you are current with Terraform practices.

Q68. What is import?

Brings existing resources under Terraform management — use terraform import resource_type.name resource_id, then write matching configuration.

Q69. What is lifecycle use case in real project?

I use prevent_destroy on Key Vault and storage accounts, and create_before_destroy for SSL certificate rotation and VM image updates for zero-downtime.

Q70. Count vs for_each?

Count creates N identical resources using a number. for_each creates resources from a map/set with unique keys — easier to add/remove specific items. for_each is preferred.

Section 7: DevSecOps — tfsec, tflint, Checkov

Q71. What is tfsec?

Static analysis security scanner for Terraform code — detects misconfigurations like open security groups, unencrypted storage, missing HTTPS, or public blob access before deployment.

Q72. What is tflint?

Terraform linter that checks for coding errors, deprecated syntax, invalid variable types, and provider-specific best practices.

Q73. What is Checkov?

Policy-as-code scanner that checks Terraform against hundreds of security and compliance rules — covering CIS benchmarks, PCI-DSS, SOC2, and HIPAA.

Q74. How do you integrate DevSecOps tools in CI/CD?

In Azure DevOps pipeline: tflint runs first for code quality, then tfsec and Checkov for security, then terraform plan only if scans pass. Critical findings block the pipeline.

■ *Tip: "Shift-left security — catching misconfigurations in the pipeline before they ever reach cloud infrastructure."*

Q75. tfsec vs Checkov difference?

tfsec focuses purely on Terraform HCL security. Checkov supports multiple IaC frameworks (Terraform, ARM, Kubernetes, Dockerfile). Running both gives broader coverage.

Section 8: CI/CD Integration

Q76. How Terraform integrates with CI/CD?

Terraform commands run in pipelines for automated infrastructure deployment with proper controls — init, scan, plan, approval, apply stages.

Q77. How do you integrate Terraform with Azure DevOps?

Azure DevOps YAML Pipeline stages: (1) terraform init, (2) tfint + tfsec + Checkov scan, (3) terraform plan with saved plan file, (4) manual approval gate for Production, (5) terraform apply using saved plan.

Q78. What is plan approval in pipeline?

Before terraform apply runs in production, the saved plan is reviewed and manually approved — prevents unintended or risky infrastructure changes from reaching production.

Q79. Blue-green deployment using Terraform?

Create new infrastructure (green), switch traffic routing via Load Balancer or Traffic Manager, validate, then destroy old infrastructure (blue) — zero-downtime deployment.

Q80. How do you ensure zero downtime deployment?

Using `create_before_destroy` lifecycle, blue-green deployment strategy, and Azure Load Balancer or Traffic Manager for traffic routing during infrastructure updates.

Section 9: Advanced Concepts

Q81. What is interpolation in Terraform?

References variables or resource values using `${}` syntax — for example: `${var.location}` or `${azurerm_resource_group.main.name}`.

Q82. What is expression in Terraform?

Defines how values are computed — includes conditions, functions, for loops, and references to other resources.

Q83. What are functions in Terraform?

Built-in functions to process data: `length()`, `lookup()`, `join()`, `toset()`, `merge()`, `format()`, `filebase64()`, `coalesce()`, and many more.

Q84. What is conditional expression?

Ternary operator: `condition ? true_value : false_value`. Example: `var.env == "prod" ? "Premium" : "Standard"`.

Q85. What is for loop in Terraform?

Used to iterate over lists or maps to transform or filter values — commonly used in local values and output expressions.

Q86. What is Dynamic Block?

Generates repeated nested configuration blocks dynamically from a list or map — for example, creating multiple NSG security rules from a variable list.

Q87. What is a Data Source?

Fetches information about existing infrastructure NOT managed by current Terraform — like reading an existing Key Vault URI or VNet ID.

Q88. What is Null Resource?

A dummy resource with no real infrastructure — used to run local or remote scripts. Generally not recommended.

Q89. What is Provisioner?

Used to execute scripts on local or remote machines during resource creation — local-exec and remote-exec are the two types.

Q90. Why provisioners are not recommended?

They break idempotency — if resource is recreated, the script runs again with potentially different results. Use Azure VM Extensions or Ansible instead.

Q91. What is Provider Alias?

Allows configuring multiple instances of the same provider — example: two azurerm configurations for different Azure regions (East US and West Europe).

Q92. Example of provider alias use?

Deploy resources in two different Azure regions using the same azurerm provider with different aliases — one for primary, one for disaster recovery.

Q93. What is Remote State Data Source?

Allows one Terraform project to read output values from another project state file — used to share VNet IDs, Key Vault URIs across separate Terraform configurations.

Q94. What is partial configuration in backend?

Splitting backend config between code and CLI flags at runtime — useful when storage account keys should not be stored in code and are passed as pipeline secrets.

Q95. What is Terraform Workspace?

Allows managing multiple environments with same configuration and separate state files. For complex production environments, separate directory structures are often preferred.

Q96. What is terraform workspace list?

Shows all available workspaces for the current configuration.

Q97. How to switch workspace?

`terraform workspace select <name>` — switches to the named workspace.

Q98. What is default workspace?

The default environment created automatically by Terraform when no other workspace is specified.

Q99. What is Terratest?

A Go-based testing framework for Terraform — allows writing automated tests that deploy real infrastructure, run assertions, and then destroy it.

Q100. How do you test Terraform code?

Using terraform validate for syntax, tflint for standards, tfsec and Checkov for security, terraform plan for dry-run, and Terratest for automated module testing.

Section 10: Terraform Cloud & Enterprise

Q101. What is Terraform Cloud?

A managed service by HashiCorp providing remote execution, state management, team collaboration, policy enforcement (Sentinel), cost estimation, and drift detection.

Q102. Terraform Cloud vs CLI?

CLI runs on local machine or CI server. Terraform Cloud provides remote execution with better collaboration, audit logs, Sentinel policy enforcement, and cost estimation.

Q103. What is Sentinel in Terraform?

HashiCorp policy-as-code framework — enforces organizational rules before infrastructure is deployed. Example: "All VMs must have encryption enabled."

Q104. What is policy as code?

Ensures infrastructure follows organizational rules automatically — checks enforced in pipeline, not left to individual engineers.

Q105. What is cost estimation in Terraform?

Terraform Cloud estimates monthly infrastructure cost before deployment — helps teams make cost-aware decisions.

Q106. What is speculative plan?

A plan run in Terraform Cloud without applying — used for reviewing changes in PR workflows before merging code.

Q107. What is run trigger?

Automatically triggers a Terraform run when a dependent workspace changes — useful for chained infrastructure dependencies.

Q108. What is drift detection in Terraform Cloud?

Automatically detects infrastructure changed outside Terraform and alerts the team — prevents drift from going unnoticed.

Q109. What is private module registry?

A secure internal registry to store and version organization-specific reusable modules — accessible only within your organization.

Q110. What is versioning in modules?

Assigning versions to modules maintains stability — existing callers are not affected by updates until they explicitly upgrade.

Section 11: Scenario & Troubleshooting Questions

Q111. If terraform apply deletes wrong resource?

Use Git version control rollback and re-apply correct configuration. Investigate how `prevent_destroy` lifecycle could prevent this in future.

Q112. If backend config changes?

Run `terraform init` again to reconfigure backend — Terraform will migrate state to the new backend location.

Q113. If provider plugin fails?

Re-run `terraform init`, check network connectivity, verify version constraints in `required_providers`, and `.terraform.lock.hcl` for conflicts.

Q114. If module is updated but not reflecting?

Run `terraform init -upgrade` to download the updated module version, then re-run `plan` and `apply`.

Q115. If resource stuck in destroying state?

Check for dependency issues, review error logs, remove from state if needed, and clean up manually in Azure portal.

Q116. If credentials expire?

Update service principal or managed identity credentials in pipeline service connection, then re-run the pipeline.

Q117. If pipeline fails but infra partially created?

Fix the root cause and re-run pipeline. Terraform checks state and only creates what is missing — partial state handled gracefully.

Q118. If two people run terraform apply at same time?

State locking prevents concurrent execution — one gets the lock, others must wait. This is why remote backend with locking is essential.

Q119. If state file is corrupted?

Restore from backup using Azure Blob Storage versioning, or re-import resources using terraform import for each affected resource.

Q120. If someone deletes resource manually?

Terraform detects drift on next terraform plan and recreates the resource on next apply — unless lifecycle rules prevent it.

Q121. If apply fails in middle?

Fix the issue and re-run terraform apply. Terraform resumes based on current state — only creates what is still missing.

Q122. If state lock is stuck?

Use terraform force-unlock <lock-id> carefully — only when certain no other Terraform operation is running.

Q123. If resource needs rename?

Use terraform state mv old_address new_address to rename in state file — avoids destroying and recreating the resource.

Q124. If you want to delete from state only?

Use terraform state rm — Terraform forgets the resource but it stays alive in Azure. NOT deleted from cloud.

Q125. How to handle large team changes?

Use CI/CD pipelines with approval process — no one applies Terraform manually. All changes go through code review and pipeline approval gates.

Q126. How to restrict access?

RBAC on Azure Blob Storage for state file, minimum permission service principal for pipeline, and PR approval process in Git.

Q127. How to audit Terraform changes?

Git history tracks all configuration changes. Pipeline logs capture every plan and apply. Azure Activity Logs record actual resource changes in Azure.

Q128. How to ensure consistency?

Use modules, remote backend, `.terraform.lock.hcl` committed to Git, and proper pipeline flow with same plan file applied in all stages.

Section 12: Real Project Answers

Q129. How do you use Terraform in your project?

I use Terraform to provision Azure resources including VMs, storage accounts, VNets, NSGs, Key Vault, and App Services. I have created reusable modules for each resource type. Terraform is integrated with Azure DevOps pipelines with manual approval gates for production. Remote state is stored in Azure Blob Storage with blob lease locking.

Q130. How do you manage multiple environments?

Using separate environment folders (dev, test, prod) each with their own backend config and state file. Variables control environment-specific values like VM sizes and instance counts.

Q131. How do you collaborate in a team?

Using remote backend for shared state, Git for version control with PR reviews, and Azure DevOps CI/CD pipelines for all deployments. Nobody applies Terraform manually — everything goes through the pipeline.

Q132. How do you secure Terraform state?

Azure Blob Storage backend with RBAC — only pipeline service principal has read/write access. Storage account has soft-delete, versioning, and encryption enabled.

Q133. How do you manage secrets in Terraform?

We use Azure Key Vault for all secrets. In pipelines, Key Vault references used as pipeline variables. Sensitive Terraform variables marked as sensitive to prevent exposure in logs.

Q134. State + Backend (project explanation)?

Terraform state tracks current infrastructure and maps it to configuration. I use Azure Blob Storage as remote backend with state locking for secure and collaborative state management.

Q135. Modules (project explanation)?

Modules are reusable blocks of Terraform code. I use modules to maintain consistency across dev, test, and production environments without repeating code.

Q136. Variables (project explanation)?

Variables make configurations flexible and reusable. I use variables to manage different values for dev, test, and production without changing core code.

Q137. CI/CD Integration (project explanation)?

Terraform is integrated with Azure DevOps pipelines where we run init, plan, and apply stages. This automates infrastructure deployment with proper approval process — no manual runs.

Q138. Drift (project explanation)?

Drift occurs when infrastructure is changed manually outside Terraform. We detect it using terraform plan and fix it by re-applying the configuration to restore desired state.

Q139. Scenario: Failure (project explanation)?

If deployment fails, I check logs and identify the issue. Fix the configuration and re-run terraform apply. If needed, rollback using previous stable configuration from Git.

Q140. Project Explanation (full)?

In my project, I use Terraform to provision Azure resources, manage CI/CD pipelines in Azure DevOps, ensure environment consistency using modules and remote state, and embed DevSecOps tools (tfsec, tflint, Checkov) in every pipeline.

Q141. Team Collaboration (full)?

We use remote backend for shared state, Git for version control, and CI/CD pipelines for controlled deployments. PR reviews required for all infrastructure changes.

Q142. What is the biggest challenge you faced in Terraform?

Managing state conflicts in a team environment. Resolved by implementing remote backend with locking and making all changes go through CI/CD pipelines — no one runs terraform apply locally in production.

■ *Tip: Show the solution, not just the problem — this demonstrates maturity and ownership.*

Q143. How do you handle failed deployment?

Analyze logs, fix the configuration issue, and re-run the pipeline. If production needs rollback, revert the Git commit and re-run — Terraform restores the previous state.

Q144. How do you ensure zero downtime deployment?

Using `create_before_destroy` lifecycle, blue-green deployment strategies, and Azure Load Balancer or Traffic Manager for traffic routing during updates.

Q145. How do you rollback Terraform changes?

Revert code changes in Git and re-run `terraform apply` — Terraform restores infrastructure to match the reverted configuration.

Q146. How do you upgrade Terraform provider?

Update version constraint in `required_providers` and run `terraform init -upgrade`. Always test with `terraform plan` before applying to production.

Q147. How do you import existing resources?

Using `terraform import resource_type.name resource_id`. Then write matching configuration and run `terraform plan` to verify no drift.

Q148. How do you ensure best practices?

Use modules for all reusable infrastructure patterns Remote backend with state locking for all environments `tfsec` + `tflint` + Checkov in every CI/CD pipeline Code reviews via pull requests before merge Saved plan files for production apply with manual approval `prevent_destroy` on critical production resources Sensitive variables for all secrets — never hardcode Azure Key Vault for secrets management Versioning and soft-delete on state storage account

Q149. What is idempotency in Terraform?

Applying the same configuration multiple times always produces the same result — if infrastructure already matches, nothing changes. Core property of declarative IaC.

Q150. What is your final tip for Terraform interviews?

Always connect answers to real experience. Explain how you use commands in your project, what problems they solved, and what you learned. Your telecom + DevOps dual background is a genuine advantage — use it confidently!

Section 13: Tricky Conceptual Questions

Q151. Difference between terraform apply and terraform push?

terraform apply applies changes locally or in CI/CD pipeline. terraform push was used in older Terraform Enterprise — now deprecated and replaced by Terraform Cloud workflows. Note: If asked, say terraform push is deprecated — shows you are up to date.

Q152. Can Terraform manage resources created manually in Azure?

Yes — using terraform import you can bring manually created resources under Terraform management. After importing, write matching HCL configuration and run terraform plan to verify no drift.

Q153. What is the difference between immutable and mutable infrastructure?

Mutable = modify existing servers in place. Immutable = replace old resources with new ones on every change (Terraform approach using `create_before_destroy`). Immutable is more predictable and reduces configuration drift.

Q154. Can you use Terraform without a state file?

No — Terraform requires state to function. Without state, it cannot track what resources exist, plan updates, or perform deletions. State is fundamental to how Terraform works.

Q155. What happens when you run terraform apply with no changes?

Terraform reports "No changes. Infrastructure is up-to-date." — confirms infrastructure is in desired state. This is idempotency in action.

Q156. What is the difference between `count = 0` and removing a resource block?

`count = 0` keeps the resource block in configuration but creates zero instances — Terraform destroys any existing resource. Removing the block achieves the same but is cleaner. `count = 0` is useful for conditional resource creation.

Q157. What is the splat expression in Terraform?

Used to access attributes of all resources created by `count` or `for_each`. Example: `resource[*].id` returns a list of all IDs — useful in outputs and other resource references.

Q158. What is the difference between toset() and tolist()?

toset() converts to a set — removes duplicates and loses ordering. tolist() converts to a list — maintains order and allows duplicates. Use toset() with for_each to ensure unique keys.

Q159. What is terraform plan -target?

Targets a specific resource for plan or apply — only that resource and dependencies are processed. Useful for debugging but NOT recommended for regular use as it can create state inconsistencies. Note: -target is for debugging only, not a recommended workflow practice.

Q160. Difference between module source from local path vs registry?

Local path (./modules/vnet) is for modules within same project. Registry source pulls from public or private registry with version pinning. Registry modules are versioned and shared across projects.

Section 14: Azure-Specific Terraform Questions

Q161. What is azurerm provider?

The official HashiCorp provider for Microsoft Azure — allows Terraform to create and manage Azure resources like VMs, VNets, storage, Key Vault, AKS, and everything else in Azure.

Q162. How do you authenticate Terraform with Azure?

Four options: (1) Azure CLI login for local dev, (2) Service Principal with client secret, (3) Service Principal with certificate, (4) Managed Identity (preferred in CI/CD). In Azure DevOps, we use Service Principal via service connection.

■ **Tip:** "In my pipeline, I use a Service Principal configured as Azure DevOps service connection — no secrets stored in code."

Q163. What is a Service Principal in Azure Terraform?

An identity in Azure Active Directory used by Terraform pipeline to authenticate with Azure. It has specific RBAC roles assigned — giving only the permissions needed to create and manage infrastructure.

Q164. What is Managed Identity and why is it preferred?

Managed Identity is an Azure AD identity assigned to Azure services that can authenticate without credentials. Preferred because there are no secrets to manage, rotate, or accidentally expose.

Q165. How do you store Azure credentials securely for Terraform?

In Azure DevOps, use service connections with Service Principal. Store client secrets in Azure Key Vault or pipeline secret variables — never in Terraform code or tfvars files.

Q166. What Azure RBAC role does Terraform service principal need?

Minimum: Contributor role on subscription or resource group. For role assignments, Owner or User Access Administrator is needed. Always apply principle of least privilege.

Q167. How do you configure Azure Blob Storage as Terraform backend?

Create a storage account and container in Azure Define backend "azurerm" block in backend.tf Specify: resource_group_name, storage_account_name, container_name, key Run terraform init to initialize the backend Enable versioning and soft-delete on storage account for backup

Q168. Difference between azurerm_resource_group and data azurerm_resource_group?

azurerm_resource_group (resource) creates a new resource group. data "azurerm_resource_group" reads an existing one NOT managed by current Terraform — used to reference its ID or location.

Q169. How do you handle Azure Key Vault secrets in Terraform?

Reference existing Key Vault secrets using data source to read values at runtime. Never store secrets in tfvars or code. Mark any Terraform variables containing secret values as sensitive = true.

Q170. What is azurerm_role_assignment in Terraform?

Assigns Azure RBAC role to a principal (user, service principal, managed identity) at a specific scope. Essential for setting up access control as part of infrastructure provisioning.

Section 15: Pipeline & Workflow Deep Dive

Q171. Walk me through your complete Terraform CI/CD pipeline.

Stage 1: Checkout code from Git. Stage 2: terraform init with backend config. Stage 3: tfllint for code quality. Stage 4: tfsec and Checkov for security scan. Stage 5: terraform plan with -out flag to save plan. Stage 6: Manual approval gate (Production). Stage 7: terraform apply using saved plan file.

■ **Tip:** "Nothing reaches production without code review, security scan, plan review, and manual approval."

Q172. How do you pass different variables per environment in pipeline?

Use separate variable files per environment (dev.tfvars, test.tfvars, prod.tfvars) and pass them using -var-file flag. Pipeline environment-specific variable groups inject sensitive values at runtime.

Q173. How do you prevent terraform destroy in pipeline?

Remove destroy stage from production pipeline. Use prevent_destroy lifecycle in code. Add pipeline branch protection so destroy can only run from specific branches with additional approvals.

Q174. How do you handle secrets in Azure DevOps Terraform pipeline?

Store secrets in Azure Key Vault and reference them as pipeline secret variables using Key Vault linking in Azure DevOps variable groups. Marked as secret in pipeline — never printed in logs.

Q175. What is the benefit of running plan in PR and apply on merge?

Running plan in PR shows infrastructure impact before code is merged — reviewers see exactly what changes. Apply runs only after code is approved and merged — safest workflow for production infrastructure.

Q176. How do you handle Terraform in monorepo with multiple environments?

Use folder structure: /infra/dev, /infra/test, /infra/prod — each with own backend config and tfvars. Pipeline detects which folder changed and only runs Terraform for that environment. Shared modules live in /infra/modules.

Q177. What is drift remediation in pipeline?

Schedule a daily pipeline job that runs terraform plan and alerts the team if drift is detected. Team then decides whether to apply (fix drift) or import (accept the change).

Q178. How do you version Terraform infrastructure changes?

Use Git tags or semantic versioning in module sources. Keep module versions pinned in calling configurations. All changes go through pull requests with meaningful commit messages for auditability.

Section 16: Comparison Questions (Interviewer Favorites)

Q179. Terraform vs Ansible?

Terraform is for provisioning infrastructure (IaC) — declarative, manages cloud resource lifecycle. Ansible is for configuration management — procedural, configures software on existing servers. In practice: Terraform creates the VM, Ansible configures it.

Q180. Terraform vs ARM Templates?

ARM Templates are Azure-specific, verbose JSON, no state management. Terraform is cloud-agnostic, cleaner HCL syntax, maintains state, and has rich module ecosystem. Terraform is preferred for multi-cloud and team environments.

Q181. Terraform vs Pulumi?

Terraform uses declarative HCL. Pulumi uses general-purpose programming languages (Python, TypeScript, Go). Terraform has larger community and module ecosystem. Pulumi offers more flexibility for developers who prefer coding in familiar languages.

Q182. Terraform vs CloudFormation?

CloudFormation is AWS-specific, uses JSON/YAML, tightly integrated with AWS. Terraform is cloud-agnostic, uses HCL, works across Azure, AWS, GCP. For multi-cloud, Terraform is the clear choice.

Q183. Declarative vs Imperative IaC?

Declarative (Terraform) = define WHAT you want, Terraform figures out HOW. Imperative (scripts) = define exactly HOW to do it step by step. Declarative is more predictable and idempotent.

Q184. Local backend vs Remote backend?

Local backend stores state on local machine — not suitable for teams (no sharing, no locking). Remote backend (Azure Blob Storage) stores state centrally — enables collaboration, locking, and backup.

Q185. terraform.tfvars vs environment variables for secrets?

terraform.tfvars should NEVER contain secrets — they are committed to Git. Use environment variables (TF_VAR_name) or pipeline secret variables for secrets. Sensitive values should come from Azure Key Vault.

Q186. count vs for_each — when to use which?

Use count for N identical resources. Use for_each when resources have unique names or configurations. for_each is almost always the better choice — easier to add/remove specific items.

Q187. One large state or multiple small states?

Multiple small states are better — each team or component owns their state. Smaller blast radius on mistakes, faster plan and apply, clearer ownership. Use remote state data sources to share outputs between states.

Q188. list vs set vs map — when to use?

list = ordered, allows duplicates. set = unordered, no duplicates — use with for_each for unique resource keys. map = key-value pairs — use when resources need unique names and configurations.

Section 17: Security & Compliance Questions

Q189. How do you ensure Terraform code is secure?

Run tfsec, tflint, and Checkov in CI/CD for every code change. Use sensitive variables for all secrets. Reference Azure Key Vault for secret values. Apply RBAC with minimum permissions on service principals. Enable state file encryption and access control.

Q190. What common security misconfigurations does tfsec catch?

Storage accounts with public blob access enabled Key Vault without soft-delete or purge protection NSG rules with unrestricted inbound access (0.0.0.0/0) VMs without disk encryption enabled Storage accounts without HTTPS-only enforcement Missing Azure Monitor diagnostic settings

Q191. How do you rotate service principal credentials?

Use Azure Key Vault to store and rotate credentials. In Azure DevOps, update service connection when credentials are rotated. Use Managed Identity where possible to eliminate credential rotation entirely.

Q192. What is principle of least privilege in Terraform?

Terraform service principal should have only minimum permissions needed to provision specific resources — no more. Avoid Owner or Global Admin. Use resource-group scoped Contributor instead of subscription-wide access where possible.

Q193. How do you prevent accidental deletion of production resources?

Use prevent_destroy lifecycle on critical resources Remove destroy stage from production pipelines Apply Azure resource locks (CanNotDelete or ReadOnly) on critical resources Require additional approvals for any destruction operations Enable soft-delete on Key Vault and storage accounts

Q194. What are Azure Resource Locks and how do they relate to Terraform?

Azure Resource Locks (CanNotDelete, ReadOnly) are Azure-native protection that prevent deletion even if Terraform tries. In Terraform, use azurerm_management_lock resource to apply locks as code.

Section 18: Real Interview Scenario Questions

Q195. If interviewer asks: Tell me about a Terraform failure you experienced?

A team member accidentally ran terraform apply without the saved plan — it picked up unintended variable changes and modified the wrong environment. After this, we enforced strict pipeline rule: all applies must use a saved plan file, and no one applies Terraform manually. We also added prevent_destroy on critical resources.

■ **Tip:** Always end with what you *LEARNED* and what process you *IMPROVED* — shows ownership and growth mindset.

Q196. If interviewer asks: How would you migrate existing infrastructure to Terraform?

Step 1: Inventory all existing resources. Step 2: Write Terraform configuration matching existing resources. Step 3: Use terraform import to bring each resource into state. Step 4: Run terraform plan to verify zero drift. Step 5: From this point, all changes go through Terraform — no more manual changes.

Q197. If interviewer asks: How do you onboard a new engineer to your Terraform project?

Provide documentation on project structure and module usage. Give read-only access initially. Walk through the pipeline flow. Ensure they understand remote backend, state locking, and PR approval process. First week: review existing code and run plan only. After review: full pipeline access.

Q198. If interviewer asks: How do you handle a breaking change in a provider upgrade?

Test provider upgrade in dev environment first. Review provider changelog for breaking changes. Update configuration to match new resource schema. Run terraform plan to see all changes. Fix issues in dev, test, then apply to production with full approval process.

Q199. If interviewer asks: Terraform or manual — which is better and why?

Terraform is always better for team environments and production. Manual changes are: not tracked in Git, not repeatable, error-prone, and create drift. Terraform provides version control, consistency, automation, auditability, and disaster recovery. Only exception is emergency hotfixes — always followed by updating Terraform code immediately after.

Q200. Final question — What makes you a strong Terraform engineer?

I have hands-on production experience provisioning and managing Azure infrastructure using Terraform across multiple environments. I have built reusable modules, set up remote state with locking, integrated tfsec, tflint, and Checkov into CI/CD pipelines, and handled real challenges like drift and state conflicts. I understand not just HOW to write Terraform code but WHY each decision matters for security, reliability, and team collaboration. My telecom + DevOps dual background is a genuine advantage — own it with confidence!

■ **Tip:** End every interview with confidence — your 10 years of combined experience is genuinely rare. Own it! ALL 200 QUESTIONS COMPLETE! All the best Dilip bhai — you have GOT this!

■ All 200 Questions Complete!

Own your 10 years of combined experience.
Walk into that interview with confidence.

— All the best, Dilip bhai —